The Sliced Sandwich Construction

A two-sided keyed slicing mechanism on $2n$ wires

local_mixing project

July 20, 2026

branch `ssg-gen-mix-clean`

Abstract

The *sliced sandwich* is a keyed obfuscation construction that computes a source function C on one designated input slice and, on the same slice, reveals only a fresh random function D^{-1} when run backwards. It is a slicing mechanism specific to the sandwich structure and is distinct from the gadget and feistel paths: it uses no secret sharing, placing C and a random padding computation D as plaintext g57 circuits on the low half of $2n$ wires, with two slice blocks of CNOT/CCNOT gates interleaved through the two computations. This note gives the construction, its slice semantics in both directions, the rationale, and its parameters.

1 Wires and structure

The transformed circuit acts on $2n$ wires in two halves of n: the first half (wires $0..n{-}1$) holds the input x, and the second half (wires $n..2n{-}1$) holds the auxiliary y — the *slice register*.

The circuit is a sandwich of two computations around a copy step:

$$A \;=\; \big[\, C \parallel S_1 \,\big] \;;\;\; N \;;\;\; \big[\, D \parallel S_2 \,\big],$$

where $\parallel$ denotes a random interleaving. The pieces:

- C — the source circuit, a g57 circuit placed directly on wires $0..n{-}1$. No secret sharing: this is a plaintext computation whose obfuscation comes from the subsequent mixing.
- D — a fresh random circuit of m g57 gates on wires $0..n{-}1$, the *same design* as the C block (target and two distinct controls per gate). Default $m = n(\log_2 n)^2$.
- N — the copy step $y \mathrel{\hat{=}} x$: the n CNOTs wire $(n{+}i) \mathrel{\hat{=}}$ wire i.
- S_1, S_2 — two independent slice blocks of s gates each (default $s = n \log_2 n$). Every gate targets the first half and reads at least one second-half wire with positive polarity:

$$\underbrace{x_i \mathrel{\hat{=}} y_j}_{\text{CNOT},\ \approx 1/3} \qquad \underbrace{x_i \mathrel{\hat{=}} x_j \wedge y_k}_{\text{CCNOT, rest}}$$

  so each block is *dead when the second half is zero.* S_1 is randomly interleaved with C, S_2 with D; the interleaving is uniform and preserves each computation's internal order (necessary for C and D to compute correctly).

2 Slice semantics

2.1 Forward: the slice reveals C

On the zero slice $y = 0$, trace the running state (a, b) from $(x, 0)$:

$$(x, 0) \xrightarrow{C\|S_1} (C(x),\, 0) \xrightarrow{N} (C(x),\, C(x)) \xrightarrow{D\|S_2} (\text{junk},\, C(x)).$$

During the first stage the second half is still 0, so every S_1 gate is dead and C computes correctly. The copy step moves $C(x)$ into the second half. In the third stage S_2 is *live* (the second half now holds $C(x) \neq 0$), but it only targets the first — junk — half, so it cannot corrupt the answer. Hence

$$\boxed{A(x, 0) = (\text{junk},\, C(x))}$$

with the answer on the second half.

2.2 Inverse: the same slice reveals D^{-1}

Every gate is an involution, so A^{-1} is the reversed gate list. On input $(p, 0)$:

$$(p, 0) \xrightarrow{(D\|S_2)^{-1}} (D^{-1}(p),\, 0) \xrightarrow{N} (D^{-1}(p),\, D^{-1}(p)) \xrightarrow{(C\|S_1)^{-1}} (\text{junk},\, D^{-1}(p)).$$

Now it is S_2 that is dead (its stage runs first, second half 0), so D^{-1} computes cleanly; and S_1 that fires (its stage sees a nonzero second half), junking the first half. Hence

$$\boxed{A^{-1}(p, 0) = (\text{junk},\, D^{-1}(p)).}$$

The construction is thus sliced on the *same* slice in both directions: each slice block guards one direction's correctness (S_1 forward, S_2 inverse) and scrambles the other direction's junk. The forward oracle gives C; the backward oracle gives only the random D^{-1}, never C^{-1}.

2.3 Off-slice

For $y \neq 0$ the second-half output is $y \oplus C'(x, y)$, where C' is the S_1-disturbed computation. The $y\oplus$ masking from the copy step guarantees the output differs from the clean answer $C(x)$ on every nonzero slice, independently of whether the disturbance C' happens to coincide with C at a given x.

3 Rationale

Why interleave rather than prepend. In the plain gadget path the slice block sits at the input port. Here the sandwich has two computations and a natural two-sided structure, and weaving each slice block *through* its computation ties the disturbance to intermediate states rather than only the boundary, which the mixing then disperses. Correctness is unaffected because the slice gates are dead on the slice regardless of their positions.

Why D mirrors C. D is drawn from the same distribution as the source blocks (random g57 gates on the low n wires) so that the forward and inverse directions are structurally indistinguishable: an adversary holding the mixed circuit sees two same-design computations and cannot tell from gate statistics which end carries the "real" function.

Why this gate vocabulary. The slice gates are positive-polarity CNOT/CCNOT — the same mixing-native vocabulary as the rest of the circuit, with no complemented gates and no polarity pattern that could betray the slice. The slice is the canonical zero vector, so nothing about it is encoded in the circuit beyond the deadness structure.

4 Parameters

symbol	meaning	default
n	wires of the source function (circuit is $2n$ wires)	—
m	gates in the random D computation	$n(\log_2 n)^2$
s	gates in each slice block S_1, S_2	$n \log_2 n$

Requires $n \geq 3$ (g57 gates need three distinct wires). Invocation:

```
sss --cnot --sliced-sandwich -n <n> --sandwich-m <m> --sandwich-s <s> ...
```

The transformed size before mixing is $|C| + m + n + 2s$ gates (source, D, copy step, two slice blocks); e.g. for $n = 8$ with a 40-gate source and defaults ($s = 24$, $m = 72$): $40+72+8+48 = 168$ gates, of which $40 + 72 = 112$ are g57.

5 Verification

The driver checks the pinned contract (the `SandwichSecond` view: the second half of the input is forced to zero and the second-half output must equal $C(x)$) after transformation and after every mixing round. Three unit tests cover slice-gate deadness, the forward slice contract with a permutation check, and the inverse's second-half bijection on the slice.

Provenance. Code: `sliced_sandwich_cnot`, `random_g57_xgates`, `sandwich_slice_gates`, `random_interleave` in `src/replace/gadgets.rs`; driver branch and `FunctionView::SandwichSecond` in `src/replace/main_mix_cnot.rs`; flags in `src/main.rs` and `src/commands/sss.rs`. Companion quick-reference: `docs/SLICED_SANDWICH.md`.